DOCUMENT 13 — ADMIN PANEL & RECIPE / INGREDIENT MANAGEMENT SPECIFICATION

1. Dokümanın Amacı

Bu doküman, Smart Chef & Pantry AI uygulamasında sistem yöneticisinin tarif ve malzeme verilerini yönetmesini sağlayacak Admin Management modülünün teknik gereksinimlerini tanımlar.

Bu modül aşağıdaki işlemleri kapsar:

- Tarif oluşturma
- Tarif görüntüleme
- Tarif güncelleme
- Tarif pasifleştirme / aktif hale getirme
- Tarif silme
- Malzeme oluşturma
- Malzeme görüntüleme
- Malzeme güncelleme
- Malzeme pasifleştirme / aktif hale getirme
- Malzeme arama
- Tarif ile malzeme ilişkilendirme
- Tarif içerisindeki malzeme miktarlarını yönetme
- Optional ingredient yönetimi
- URL doğrulama
- Veri bütünlüğünün korunması
- Admin authorization
- Audit/logging altyapısına uygun yapı

Bu doküman Document 04 — Database Design, Document 05 — REST API Specification, Document 08 — Flask Backend Architecture, Document 09 — Recipe Matching & Recommendation Algorithm Specification, Document 10 — Authentication & Authorization Specification ve Document 11 — Ingredient & Recipe Data Management Specification ile uyumlu olarak hazırlanmıştır.

---

2. Admin Modülünün MVP İçindeki Yeri

Admin Management modülü MVP kapsamında yer alır.

MVP'de admin tarafından:

Recipe CRUD
Ingredient CRUD
Recipe ↔ Ingredient management
Active / Inactive management
Validation

desteklenmelidir.

Ancak aşağıdaki özellikler MVP kapsamında değildir:

Advanced analytics dashboard
User behavior analytics
Automatic recipe generation
Automatic web scraping
AI-generated recipes
Bulk marketplace synchronization
Advanced role management
Multi-admin approval workflow

---

3. Admin Aktörü

Sistemde üç temel kullanıcı rolü bulunur:

Guest
Registered User
Admin

Admin, Registered User özelliklerine ek olarak yönetim endpointlerine erişebilir.

Önemli kural:

Admin yetkisi frontend tarafından belirlenmez.

Admin kontrolü backend tarafından yapılmalıdır.

---

4. Admin Authorization

Bir kullanıcının admin olup olmadığı JWT veya backend'deki kullanıcı rolü üzerinden belirlenmelidir.

Önerilen JWT payload:

```
{
  "sub": "user-uuid",
  "role": "admin"
}
```

Backend:

JWT
↓
current_user
↓
role
↓
admin?

kontrolü yapar.

Normal kullanıcı:

role = user

ise:

POST /api/admin/recipes

gibi bir endpoint'e erişememelidir.

Response:

```
{
  "success": false,
  "error": {
    "code": "ADMIN_REQUIRED",
    "message": "Admin access is required.",
    "details": {}
  }
}
```

HTTP status:

403 Forbidden

olmalıdır.

---

5. Admin Yetkisinin Frontend'de Kontrol Edilmesi

Flutter tarafında admin kullanıcılara Admin ekranı gösterilip normal kullanıcılardan gizlenebilir.

Ancak bu yalnızca UI davranışıdır.

Örneğin:

Profile
↓
if role == admin
↓
Admin Panel

kontrolü yapılabilir.

Fakat kötü niyetli kullanıcı API'yi doğrudan çağırabileceğinden backend yine:

JWT + role

kontrolünü gerçekleştirmelidir.

Frontend'deki:

isAdmin == true

tek başına güvenlik mekanizması değildir.

---

6. Admin Modülünün Temel Sorumlulukları

Admin modülü dört ana alana ayrılır:

1. Recipe Management
2. Ingredient Management
3. Recipe Ingredient Management
4. Data Validation & Integrity

---

7. Recipe Management

Admin aşağıdaki işlemleri yapabilmelidir:

Create Recipe
Get Recipe
Update Recipe
Deactivate Recipe
Activate Recipe
Delete Recipe
List Recipes
Search Recipes

Ancak fiziksel silme işlemi kontrollü kullanılmalıdır.

Önerilen yaklaşım:

Active
Inactive

durumlarının kullanılmasıdır.

---

8. Recipe Veri Modeli

Recipe tablosu:

recipes
---
id UUID PRIMARY KEY
title VARCHAR NOT NULL
description TEXT
image_url TEXT
instructions TEXT NOT NULL

source_url TEXT
youtube_url TEXT
category VARCHAR
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL

İlerleyen aşamada:

is_active BOOLEAN

alanının bulunması önerilir.

Bu alan sayesinde tarif fiziksel olarak silinmeden sistemden kaldırılabilir.

---

9. Recipe Oluşturma

Endpoint:

POST /api/admin/recipes

Authentication:

Required

Authorization:

Admin only

Örnek request:

```
{
  "title": "Tavuklu Makarna",
  "description": "Pratik tavuklu makarna tarifi.",
  "image_url": "https://example.com/chicken-pasta.jpg",
  "instructions": "Makarnayı haşlayın...",
  "source_url": "https://example.com/recipe/chicken-pasta",
  "youtube_url": "https://youtube.com/watch?v=example",
  "category": "Ana Yemek",
  "ingredients": [
    {
      "ingredient_id": "ingredient-uuid-1",
```

```
      "quantity": 200,
      "unit": "gram",
      "display_quantity": "200 gram",
      "is_optional": false
    },
    {
      "ingredient_id": "ingredient-uuid-2",
      "quantity": 100,
      "unit": "ml",
      "display_quantity": "100 ml",
      "is_optional": false
    }
  ]
}
```


---

10. Recipe Oluşturma İş Akışı

Admin Panel
↓
Create Recipe Form
↓
Client Validation
↓
POST /api/admin/recipes
↓
JWT Validation
↓
Admin Authorization
↓
Recipe Schema Validation
↓
RecipeService
↓
Ingredient Validation
↓
Recipe Creation
↓
RecipeIngredient Creation
↓
Transaction Commit
↓
Response

Recipe ve RecipeIngredient kayıtları tek bir transaction içerisinde oluşturulmalıdır.

---

11. Recipe Validation

Recipe oluşturulurken:

Title

Zorunludur.

Boş olamaz:

""

ve yalnızca whitespace içermemelidir.

Instructions

Zorunludur.

En az bir açıklayıcı içerik bulunmalıdır.

Ingredients

Tarifin en az bir:

required ingredient

içermesi önerilir.

Yalnızca optional ingredient içeren bir tarif recommendation engine açısından geçerli kabul edilmemelidir.

Category

Kontrollü bir değer olmalıdır.

Örneğin:

Ana Yemek

Çorba
Tatlı
Kahvaltı
Salata
Atıştırmalık
İçecek

gibi değerler kullanılabilir.

Kategori listesi ileride genişletilebilir.

---

12. Recipe Image URL

MVP'de tarif görselleri PostgreSQL içerisinde binary olarak saklanmamalıdır.

Database:

image_url

saklar.

Örneğin:

https://example.com/images/tavuklu-makarna.jpg

Flutter:

CachedNetworkImage

veya uygun image widget kullanarak görseli yükler.

---

13. Source URL

`source_url` tarifin orijinal kaynağını temsil eder.

Örneğin:

https://example.com/tarif/tavuklu-makarna

Admin tarafından girilen URL doğrulanmalıdır.

Geçersiz URL:

not-a-url

kabul edilmemelidir.

Alan MVP'de nullable olabilir.

---

14. YouTube URL

`youtube_url` tarifin ilişkili YouTube videosunu temsil eder.

Alan nullable olabilir.

Admin:

https://www.youtube.com/watch?v=...

veya geçerli YouTube URL formatlarından birini girebilir.

Backend URL formatını temel seviyede doğrulamalıdır.

Flutter tarafında video:

url_launcher

ile açılabilir.

Uygulama içerisinde YouTube oynatıcı oluşturmak MVP için zorunlu değildir.

---

15. Recipe Güncelleme

Endpoint:

PUT /api/admin/recipes/{recipe_id}

veya kısmi güncelleme stratejisi tercih edilirse:

PATCH /api/admin/recipes/{recipe_id}

kullanılabilir.

MVP için Recipe CRUD işlemlerinde PUT kullanılabilir.

Örnek:

PUT /api/admin/recipes/recipe-uuid

Request:

```
{
  "title": "Kremalı Tavuklu Makarna",
  "description": "Güncellenmiş tarif açıklaması.",
  "image_url": "https://example.com/new-image.jpg",
  "instructions": "Güncellenmiş hazırlama adımları...",
  "source_url": "https://example.com/recipe",
  "youtube_url": "https://youtube.com/watch?v=example",
  "category": "Ana Yemek",
  "ingredients": []
}
```

---

16. Recipe Güncellemede Transaction

Recipe güncelleme sırasında:

Recipe
+
RecipeIngredients

birlikte güncelleniyorsa tek transaction içerisinde gerçekleştirilmelidir.

Örneğin:

Recipe update
↓
Ingredient relation update
↓
Validation error
↓
ROLLBACK

olmalıdır.

Böylece yalnızca tarifin bir kısmının güncellenmesi engellenir.

---

17. Recipe Ingredient İlişkisi

Recipe ile Ingredient arasında doğrudan many-to-many ilişki yerine:

recipes
ingredients
recipe_ingredients

yapısı kullanılmalıdır.

Recipe
  │
  │ 1
  ↓
RecipeIngredient
  ↑
  │ N
Ingredient

Bu yapı sayesinde:

- quantity
- unit
- display_quantity
- is_optional

gibi tarif özelindeki bilgiler saklanabilir.

---

18. recipe_ingredients Veri Modeli

recipe_ingredients
---
id UUID PRIMARY KEY
recipe_id UUID NOT NULL FK recipes.id
ingredient_id UUID NOT NULL FK ingredients.id
quantity DECIMAL NULL

unit VARCHAR NULL
display_quantity VARCHAR NULL
is_optional BOOLEAN NOT NULL DEFAULT FALSE

Önerilen constraint:

UNIQUE(recipe_id, ingredient_id)

---

19. Aynı Ingredient'ın Tekrarlanması

Aynı tarif içerisinde:

Domates
Domates

şeklinde iki ayrı ingredient relationship bulunmamalıdır.

Örneğin:

Recipe A
├── Domates 2 adet
└── Domates 3 adet

yerine:

Recipe A
└── Domates 5 adet

veya tarifin gerçek kullanımına uygun tek bir ingredient kaydı kullanılmalıdır.

Aynı ingredient'ın tarif içerisinde iki kez bulunması:

VALIDATION_ERROR

oluşturmalıdır.

---

20. Ingredient Management

Admin aşağıdaki işlemleri yapabilir:

Create Ingredient
List Ingredients
Search Ingredient
Get Ingredient
Update Ingredient
Deactivate Ingredient
Activate Ingredient
Delete Ingredient

---

21. Ingredient Veri Modeli

ingredients
---
id UUID PRIMARY KEY
name VARCHAR NOT NULL
normalized_name VARCHAR NOT NULL
category VARCHAR
created_at TIMESTAMP NOT NULL
updated_at TIMESTAMP NOT NULL

Gelecekte:

is_active BOOLEAN

alanı eklenebilir.

---

22. Ingredient Oluşturma

Endpoint:

POST /api/admin/ingredients

Request:

```
{
  "name": "Domates",
  "category": "Sebze"
```

}

Backend:

name
↓
trim
↓
normalize
↓
duplicate check
↓
create

işlemlerini yapar.

---

23. normalized_name

`normalized_name`, ingredient eşleştirmesinde kullanılacak normalize edilmiş değerdir.

Örneğin:

" Domates "

normalize edildiğinde:

"domates"

olabilir.

Ama:

Domates
Çeri Domates
Domates Sosu

otomatik olarak aynı ingredient olarak kabul edilmemelidir.

---

24. Semantic Variant Yönetimi

Aşağıdaki değerler otomatik olarak birleştirilmemelidir:

Domates
Çeri Domates
Domates Sosu

Bunun nedeni bunların gerçek kullanım açısından farklı malzemeler olabilmesidir.

Gelecekte:

ingredient_aliases

tablosu kullanılabilir.

Örneğin:

canonical ingredient:
Domates

alias:
Tomato

gibi kontrollü ilişkiler kurulabilir.

---

25. Ingredient Duplicate Validation

Admin:

Domates

oluşturmak isterse ve sistemde:

normalized_name = "domates"

zaten mevcutsa yeni kayıt oluşturulmamalıdır.

Response:

{

```
  "success": false,
  "error": {
    "code": "INGREDIENT_ALREADY_EXISTS",
    "message": "Ingredient already exists.",
    "details": {}
  }
}
```

HTTP:

409 Conflict

---

26. Ingredient Search

Admin panelinde ingredient araması bulunmalıdır.

Endpoint:

GET /api/ingredients?search=dom

Bu endpoint normal kullanıcılar için de kullanılabilir.

Admin paneli aynı endpoint'i ingredient seçim ekranında kullanabilir.

Örneğin:

dom
↓
Domates
Domates Salçası
Çeri Domates

sonuçları döndürülebilir.

---

27. Ingredient Pagination

Ingredient sayısı arttığında bütün malzemelerin tek response içerisinde dönmesi önerilmez.

Örneğin:

GET /api/ingredients?page=1&limit=20

kullanılabilir.

Response:


```
{
  "success": true,
  "data": {
    "items": [],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 100,
      "total_pages": 5
    }
  },
  "message": "Ingredients retrieved successfully."
}
```


---

28. Recipe Listeleme — Admin

Admin için:

GET /api/admin/recipes

endpoint'i kullanılabilir.

Query parametreleri:

search
category
is_active
page
limit

Örneğin:

GET /api/admin/recipes?search=tavuk&category=Ana%20Yemek&is_active=true&page=1&limit=20

---

29. Recipe Detail — Admin

GET /api/admin/recipes/{recipe_id}

endpoint'i admin panelinde düzenleme ekranını doldurmak için kullanılabilir.

Response:


```
{
  "success": true,
  "data": {
    "id": "recipe-uuid",
    "title": "Tavuklu Makarna",
    "description": "Pratik tarif.",
    "image_url": "https://example.com/image.jpg",
    "instructions": "Makarnayı haşlayın...",
    "source_url": "https://example.com/recipe",
    "youtube_url": "https://youtube.com/watch?v=example",
    "category": "Ana Yemek",
    "is_active": true,
    "ingredients": [
      {
        "ingredient_id": "ingredient-uuid",
        "name": "Tavuk",
        "quantity": 200,
        "unit": "gram",
        "display_quantity": "200 gram",
        "is_optional": false
      }
    ]
  },
  "message": "Recipe retrieved successfully."
}
```


---

30. Recipe Active / Inactive

Tarifi fiziksel olarak silmek yerine pasifleştirmek tercih edilmelidir.

Örneğin:

is_active = false

olan tarif:

- Normal kullanıcıya önerilmez.
- Recipe Results içerisinde gösterilmez.
- Recommendation Engine tarafından değerlendirilmez.
- Ancak admin panelinde görüntülenebilir.

Bu sayede geçmiş veriler korunur.

---

31. Recipe Deactivation

Endpoint:

DELETE /api/admin/recipes/{recipe_id}

kullanıldığında sistem tasarımına göre fiziksel silme yerine:

is_active = false

uygulanabilir.

Bu nedenle endpoint'in gerçek davranışı dokümantasyonda açıkça belirtilmelidir.

Önerilen davranış:

DELETE /api/admin/recipes/{id}
↓
Soft Delete
↓
is_active = false

---

32. Recipe Activation

Pasif tarif tekrar kullanılabilir hale getirilebilir.

Örneğin:

PATCH /api/admin/recipes/{recipe_id}/activate

Request body gerektirmeyebilir.

Başarılı response:

```
{
  "success": true,
  "data": {
    "id": "recipe-uuid",
    "is_active": true
  },
  "message": "Recipe activated successfully."
}
```


---

33. Ingredient Active / Inactive

Ingredient için de benzer yapı kullanılabilir.

is_active = false

olan ingredient:

- Yeni tariflere eklenmemeli.
- Normal ingredient selection içerisinde gösterilmemeli.
- Ancak geçmiş recipe relationship kayıtları korunabilir.

Bu özellikle veri bütünlüğü açısından önemlidir.

---

34. Ingredient Silme Politikası

Bir ingredient herhangi bir recipe ile ilişkiliyse fiziksel olarak silinmesi risklidir.

Örneğin:

Domates
↓
Tavuklu Yemek
↓
RecipeIngredient

ilişkisi mevcutken Domates kaydının silinmesi:

RecipeIngredient

kaydını geçersiz hale getirebilir.

Bu nedenle ilişkili ingredient için:

Soft Delete / Deactivate

tercih edilmelidir.

---

35. Recipe Silme Politikası

Bir tarif kullanıcılar tarafından favorilenmiş olabilir.

Örneğin:

Recipe
↓
Favorite

ilişkisi varsa fiziksel silme:

- Favorite kayıtlarını etkileyebilir.
- Referans bütünlüğünü bozabilir.
- Kullanıcı geçmişini kaybettirebilir.

Bu nedenle recipe için de soft delete/deactivation önerilir.

---

36. Recipe Update ve Favorites

Tarif güncellendiğinde:

recipe_id

değişmemelidir.

Örneğin:

Recipe ID:
abc-123

aynı kalırken:

title
description
image
instructions
ingredients

güncellenebilir.

Böylece:

Favorite
↓
recipe_id = abc-123

ilişkisi bozulmaz.

---

37. Recipe Update ve Recommendation Engine

Recipe'nin ingredient ilişkileri değiştirildiğinde Recommendation Engine otomatik olarak yeni ilişkilere göre çalışmalıdır.

Örneğin:

Eski tarif:

Tavuk
Makarna

Yeni tarif:

Tavuk
Makarna
Krema

olduğunda kullanıcı:

Tavuk + Makarna

seçerse match percentage yeni ingredient listesine göre hesaplanmalıdır.

Admin değişikliği sonrasında ayrı bir model retraining gerekmez çünkü MVP matching deterministic olarak çalışmaktadır.

---

38. Recipe Ingredient Validation

RecipeIngredient oluşturulurken:

- Ingredient ID geçerli olmalı.
- Ingredient aktif olmalı.
- Quantity negatif olmamalı.
- Aynı ingredient tekrar eklenmemeli.
- Unit uygun formatta olmalı.
- `is_optional` boolean olmalı.

Örneğin:

```
{
  "ingredient_id": "invalid-id",
  "quantity": -5,
  "unit": "gram",
  "is_optional": false
}
```

geçersizdir.

---

39. Optional Ingredient

RecipeIngredient:

is_optional = true

olabilir.

Örneğin:

Karabiber

opsiyonel olabilir.

Recommendation Engine:

required ingredient

sayısını hesaplayarak match percentage oluşturur.

Optional ingredient eksikse kullanıcıya gösterilebilir ancak match percentage düşürülmez.

Bu kural Document 09 ile aynıdır.

---

40. Recipe Oluşturma Örneği

Admin:

Başlık:
Tavuklu Makarna

Kategori:
Ana Yemek

Malzemeler:

Tavuk	200 gram	Required
Makarna	250 gram	Required
Krema	100 ml	Required
Karabiber	1 çay kaşığı	Optional

Backend'de:

```
recipes
   ↓
recipe_ingredients
    ├── Tavuk
    ├── Makarna
    ├── Krema
    └── Karabiber
```

oluşturulur.

---

41. Admin Recipe Form

Flutter Admin Recipe Form:

Başlık
[_____]

Açıklama
[_____]

Kategori
[Ana Yemek ▼]

Görsel URL
[_____]

Kaynak URL
[_____]

YouTube URL
[_____]

Talimatlar
[_____]
[_____]

Malzemeler
_____

Tavuk
Miktar: 200
Birim: gram
[Required]

Makarna
Miktar: 250
Birim: gram
[Required]

Karabiber
Miktar: 1
Birim: çay kaşığı
[Optional]

[+ Malzeme Ekle]

[Kaydet]

---

42. Admin Ingredient Form

Malzeme Adı
[_____]

Kategori
[_____]

[Kaydet]

`normalized_name` kullanıcıdan ayrıca alınmamalıdır.

Backend tarafından oluşturulmalıdır.

Örneğin:

Input:
" Domates "

Backend:
name = "Domates"
normalized_name = "domates"

---

43. Flutter Admin Architecture

Önerilen yapı:

```
lib/
└── features/
    └── admin/
        ├── data/
        │   ├── datasources/
        │   │   └── admin_remote_data_source.dart
        │   ├── models/
        │   │   ├── admin_recipe_model.dart
        │   │   └── admin_ingredient_model.dart
        │   └── repositories/
        │       └── admin_repository_impl.dart
        │
        ├── domain/
        │   ├── entities/
```

```
│   ├── repositories/
│   │   └── admin_repository.dart
│   └── usecases/
│       ├── create_recipe.dart
│       ├── update_recipe.dart
│       ├── delete_recipe.dart
│       ├── create_ingredient.dart
│       ├── update_ingredient.dart
│       └── delete_ingredient.dart
│
└── presentation/
    ├── pages/
    │   ├── admin_dashboard_page.dart
    │   ├── recipe_management_page.dart
    │   ├── recipe_form_page.dart
    │   ├── ingredient_management_page.dart
    │   └── ingredient_form_page.dart
    ├── widgets/
    └── providers/
```

---

44. Admin Dashboard

MVP'de dashboard oldukça basit tutulabilir.

Önerilen yapı:

Admin Panel
————————————————

[Tarif Yönetimi]

[Malzeme Yönetimi]

[Yeni Tarif]

[Yeni Malzeme]

İleri aşamada:

Total Recipes
Active Recipes
Inactive Recipes
Total Ingredients

gibi istatistikler eklenebilir.

Ancak bunlar MVP'nin çekirdek fonksiyonu değildir.

---

45. Admin Recipe List

Önerilen görünüm:

Tarifler

[Ara...]

Tavuklu Makarna
Ana Yemek
Aktif
[Düzenle] [Pasifleştir]

Mercimek Çorbası
Çorba
Aktif
[Düzenle] [Pasifleştir]

Admin pasif tarifleri de filtreleyebilmelidir.

---

46. Admin Ingredient List

Malzemeler

[Ara...]

Domates
Sebze
Aktif
[Düzenle]

Makarna
Temel Gıda
Aktif
[Düzenle]

Eski Malzeme

Diğer
Pasif
[Aktif Et]

---

47. Admin Route Protection

Flutter router:

/admin
/admin/recipes
/admin/recipes/create
/admin/recipes/:id/edit
/admin/ingredients
/admin/ingredients/create
/admin/ingredients/:id/edit

gibi route'lar içerebilir.

Router seviyesinde:

authenticated?
+
admin?

kontrolü yapılabilir.

Ancak tekrar belirtmek gerekir:

Router guard ≠ Backend security

Backend authorization zorunludur.

---

48. Admin API Routes

Önerilen endpointler:

Recipes

GET /api/admin/recipes
GET /api/admin/recipes/{recipe_id}

POST   /api/admin/recipes
PUT   /api/admin/recipes/{recipe_id}
DELETE /api/admin/recipes/{recipe_id}
PATCH /api/admin/recipes/{recipe_id}/activate

Ingredients

GET   /api/admin/ingredients
GET   /api/admin/ingredients/{ingredient_id}
POST   /api/admin/ingredients
PUT   /api/admin/ingredients/{ingredient_id}
DELETE /api/admin/ingredients/{ingredient_id}
PATCH /api/admin/ingredients/{ingredient_id}/activate

Normal kullanıcı için ingredient search:

GET /api/ingredients

şeklinde kalabilir.

---

49. Admin Service Architecture

Backend:

routes/admin_routes.py
↓
schemas
↓
Admin / Recipe / Ingredient Services
↓
repositories
↓
SQLAlchemy
↓
PostgreSQL

Mevcut servis yapısına göre:

RecipeService
IngredientService

CRUD business logic'lerini taşıyabilir.

`admin_routes.py` yalnızca admin endpointlerini expose eder.

---

50. Admin Decorator

Örneğin backend'de:

@jwt_required()
@admin_required

gibi bir authorization yapısı kullanılabilir.

Mantık:

JWT valid?
↓
Yes
↓
Current user
↓
Role == admin?
↓
Yes → Continue
No → 403

Admin kontrolü merkezi bir decorator veya authorization service içerisinde tutulmalıdır.

---

51. Schema Responsibilities

Recipe schema:

- title validation
- description validation
- URL validation
- category validation
- ingredient payload validation

Ingredient schema:

- name validation
- category validation

RecipeIngredient schema:

- ingredient ID UUID validation
- quantity validation
- unit validation
- optional boolean validation

Schema business logic gerçekleştirmemelidir.

Örneğin schema:

"ingredient database'de var mı?"

sorusuna cevap vermemelidir.

Bu service katmanının sorumluluğudur.

---

52. RecipeService Responsibilities

`RecipeService`:

create_recipe()
get_recipe()
update_recipe()
deactivate_recipe()
activate_recipe()
list_recipes()
validate_recipe_ingredients()

gibi işlemleri yönetir.

Service:

- Recipe'nin geçerliliğini kontrol eder.
- Ingredient ilişkilerini kontrol eder.
- Transaction yönetimini koordine eder.
- Repository'leri kullanır.

---

53. IngredientService Responsibilities

`IngredientService`:

create_ingredient()
get_ingredient()
update_ingredient()
deactivate_ingredient()
activate_ingredient()
list_ingredients()
search_ingredients()

işlemlerini yönetir.

Ayrıca:

normalized_name
duplicate detection

işlemleri burada gerçekleştirilir.

---

54. Repository Responsibilities

RecipeRepository:

create
get_by_id
list
search
update
deactivate
activate

IngredientRepository:

create
get_by_id
get_by_normalized_name
list
search
update
deactivate

activate

RecipeIngredientRepository:

create
create_many
get_by_recipe
delete_by_recipe
replace_recipe_ingredients

Repository yalnızca database erişiminden sorumlu olmalıdır.

---

55. Recipe Update Strategy

Recipe ingredient listesi güncellenirken iki yaklaşım mümkündür:

Yaklaşım A — Diff

Mevcut ilişkiler karşılaştırılır:

Added
Removed
Updated

ve yalnızca değişen kayıtlar güncellenir.

Yaklaşım B — Replace

Mevcut RecipeIngredient kayıtları silinir ve yeni liste oluşturulur.

MVP için:

Replace + Transaction

yaklaşımı daha basit ve anlaşılır olabilir.

Örneğin:

BEGIN
↓
Update Recipe
↓

Delete old RecipeIngredients
↓
Create new RecipeIngredients
↓
COMMIT

Herhangi bir hata olursa:

ROLLBACK

uygulanır.

---

56. Data Integrity

Admin paneli veri oluşturduğu için veri kalitesi sistemin recommendation başarısını doğrudan etkiler.

Örneğin:

Domates
domates
DOMATES

gibi farklı kayıtların kontrolsüz oluşturulması ingredient matching sistemini bozabilir.

Bu nedenle ingredient normalization zorunlu olmalıdır.

Benzer şekilde:

Tavuklu Makarna
Tavuklu Makarna

gibi duplicate recipe kontrolü de uygulanabilir.

Recipe title duplicate kontrolü MVP'de zorunlu değildir ancak admin'e uyarı verilmesi önerilir.

---

57. Recipe Data Quality

Her recipe için minimum:

title
instructions
at least one required ingredient

bulunmalıdır.

Önerilen:

description
image_url
category
source_url
youtube_url

alanları da mümkün olduğunca doldurulmalıdır.

Özellikle:

image_url

Recipe Results ve Recipe Detail kullanıcı deneyimi açısından önemlidir.

---

58. Source Data Policy

Recipe verileri sisteme kontrolsüz scraping ile otomatik aktarılmamalıdır.

MVP'de admin:

Verified / approved source

üzerinden tarif verisini sisteme ekleyebilir.

Kaynağın kullanım koşulları ve içerik lisansları ayrıca dikkate alınmalıdır.

Bu sistem teknik olarak URL saklayabilir ancak içerik kullanım hakkını otomatik olarak garanti etmez.

---

59. Admin Validation Hataları

Örneğin title boşsa:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Recipe title is required.",
    "details": {
      "field": "title"
    }
  }
}
```

Ingredient listesi boşsa:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Recipe must contain at least one required ingredient.",
    "details": {
      "field": "ingredients"
    }
  }
}
```

---

60. Admin UI Error Handling

Backend hata mesajı kullanıcıya doğrudan teknik haliyle gösterilmemelidir.

Örneğin:

INGREDIENT_ALREADY_EXISTS

Flutter tarafından:

Bu malzeme zaten kayıtlı.

şeklinde gösterilebilir.

Benzer şekilde:

RECIPE_NOT_FOUND

→

Tarif bulunamadı.

olabilir.

---

61. Admin Loading State

Recipe oluştururken:

[Kaydet]

butonu:

[Kaydediliyor...]

haline gelebilir.

Aynı anda ikinci submit engellenmelidir.

Bu duplicate request riskini azaltır.

---

62. Form Unsaved Changes

Recipe edit ekranında admin değişiklik yapıp geri dönmek isterse:

Değişiklikleriniz kaydedilmedi.
Çıkmak istediğinizden emin misiniz?

uyarısı gösterilebilir.

Bu özellik veri kaybını azaltır.

MVP'de basit bir dirty-form kontrolü yeterlidir.

---

63. Admin Recipe Delete Confirmation

Soft delete uygulanacaksa:

Tarifi Pasifleştir

aksiyonu öncesinde:

Bu tarifi pasifleştirmek istediğinizden emin misiniz?
Tarif kullanıcı önerilerinde artık gösterilmeyecek.

şeklinde onay alınabilir.

---

64. Ingredient Deactivation Confirmation

Bu malzemeyi pasifleştirmek istediğinizden emin misiniz?

Malzeme mevcut tarif ilişkilerinden silinmeyecek,
ancak yeni seçimlerde kullanılamayacaktır.

Bu davranış veri bütünlüğünü korur.

---

65. Normal Kullanıcıya Etkisi

Admin yeni recipe eklediğinde:

Admin
↓
Create Recipe
↓
Database
↓
RecipeService
↓
Recommendation Engine

sonrasında tarif otomatik olarak recommendation sisteminin veri kümesine dahil olur.

Ek bir model eğitimi gerekmez.

Çünkü MVP algoritması database'deki aktif recipe + ingredient ilişkileri üzerinden deterministic olarak çalışır.

---

66. Inactive Recipe Recommendation Davranışı

Eğer:

recipe.is_active = false

ise Recommendation Engine bu tarifi hesaba katmamalıdır.

Filtreleme:

WHERE recipes.is_active = true

mantığıyla gerçekleştirilmelidir.

Böylece pasif tarif yanlışlıkla kullanıcıya önerilmez.

---

67. Inactive Ingredient Davranışı

`ingredient.is_active = false` olduğunda:

- Ingredient selection'da gösterilmemelidir.
- Yeni recipe oluşturulurken seçilememelidir.
- Yeni cart item olarak eklenmesi engellenebilir.

Ancak geçmiş recipe ilişkileri korunabilir.

Bu sayede eski veriler bozulmaz.

---

68. Admin ve Cart İlişkisi

Admin bir ingredient'ı pasifleştirdiğinde mevcut kullanıcı sepetindeki kayıtlar otomatik olarak silinmemelidir.

Örneğin:

User Cart:
Domates 5 adet

Admin:

Domates → inactive

yaparsa mevcut cart kaydı korunabilir.

Kullanıcı sepetini görebilir.

Ancak yeni cart ekleme sırasında pasif ingredient kabul edilmeyebilir.

Bu davranış MVP'de açıkça tanımlanmalıdır.

---

69. Admin ve Favorites İlişkisi

Recipe inactive olduğunda kullanıcının favorite kaydı otomatik olarak silinmemelidir.

Örneğin:

Favorite:
Tavuklu Makarna

Recipe inactive olursa favorite relationship korunabilir.

Favorites ekranında:

Tarif artık kullanılamıyor.

uyarısı gösterilebilir veya inactive tarifler kullanıcı favorilerinden filtrelenebilir.

MVP için önerilen yaklaşım:

Inactive recipe
↓
Normal recommendation'da görünmez
↓

Favorite relationship korunur

Recipe tekrar aktif edilirse favori ilişkisi yeniden kullanılabilir.

---

70. Audit Logging

MVP'de tam kapsamlı audit paneli zorunlu değildir.

Ancak backend loglarında kritik admin işlemlerinin kaydedilmesi önerilir:

admin_user_id
action
resource_type
resource_id
timestamp

Örneğin:

admin_user_id = ...
action = UPDATE_RECIPE
resource_type = recipe
resource_id = ...
timestamp = ...

Password, JWT, social token veya başka hassas bilgiler loglanmamalıdır.

---

71. Future Audit Table

İleride:

admin_audit_logs

tablosu eklenebilir.

Örneğin:

id
admin_user_id
action

resource_type
resource_id
old_value
new_value
created_at

Ancak `old_value` ve `new_value` alanlarında hassas veri tutulmamalıdır.

---

72. Test Senaryoları — Admin Authorization

Test 1

Guest
→ POST /api/admin/recipes
→ 401

Test 2

Registered User
→ POST /api/admin/recipes
→ 403

Test 3

Admin
→ POST /api/admin/recipes
→ 201

Test 4

Expired JWT
→ Admin endpoint
→ TOKEN_EXPIRED

---

73. Test Senaryoları — Ingredient

Test 1

Geçerli ingredient oluşturma:

Domates
→ Success

Test 2

Duplicate:

Domates
Domates
→ INGREDIENT_ALREADY_EXISTS

Test 3

Whitespace:

" Domates "
→ normalized_name = "domates"

Test 4

Invalid name:

""
→ VALIDATION_ERROR

Test 5

Inactive ingredient:

is_active = false
→ New recipe relation blocked

---

74. Test Senaryoları — Recipe

Test 1

Geçerli recipe:

Title
Instructions
Required Ingredient
→ Success

Test 2

No required ingredient:

Only optional ingredients
→ Validation Error

Test 3

Invalid ingredient:

Unknown ingredient_id
→ INGREDIENT_NOT_FOUND

Test 4

Duplicate ingredient:

Tomato
Tomato
→ Validation Error

Test 5

Negative quantity:

quantity = -5
→ Validation Error

Test 6

Inactive recipe:

is_active = false
→ Excluded from recommendation

---

75. Test Senaryoları — Recipe Update

Başlangıç:

Recipe:
Tavuk
Makarna

Update:

Tavuk
Makarna
Krema

Beklenen:

Old relationships replaced
New relationships created
Transaction committed

Bir ingredient geçersizse:

Transaction rollback

ve eski veri korunmalıdır.

---

76. Flutter Admin Tests

Widget testleri:

AdminDashboard
RecipeList
RecipeForm
IngredientList
IngredientForm

için yazılabilir.

Recipe Form:

- Required fields
- Ingredient add
- Ingredient remove
- Optional toggle
- Save button
- Validation
- Loading
- Error

test edilmelidir.

---

77. API Integration Tests

Aşağıdaki endpointler test edilmelidir:

GET /api/admin/recipes
GET /api/admin/recipes/{id}
POST /api/admin/recipes
PUT /api/admin/recipes/{id}
DELETE /api/admin/recipes/{id}
PATCH /api/admin/recipes/{id}/activate

GET /api/admin/ingredients
GET /api/admin/ingredients/{id}
POST /api/admin/ingredients
PUT /api/admin/ingredients/{id}
DELETE /api/admin/ingredients/{id}
PATCH /api/admin/ingredients/{id}/activate

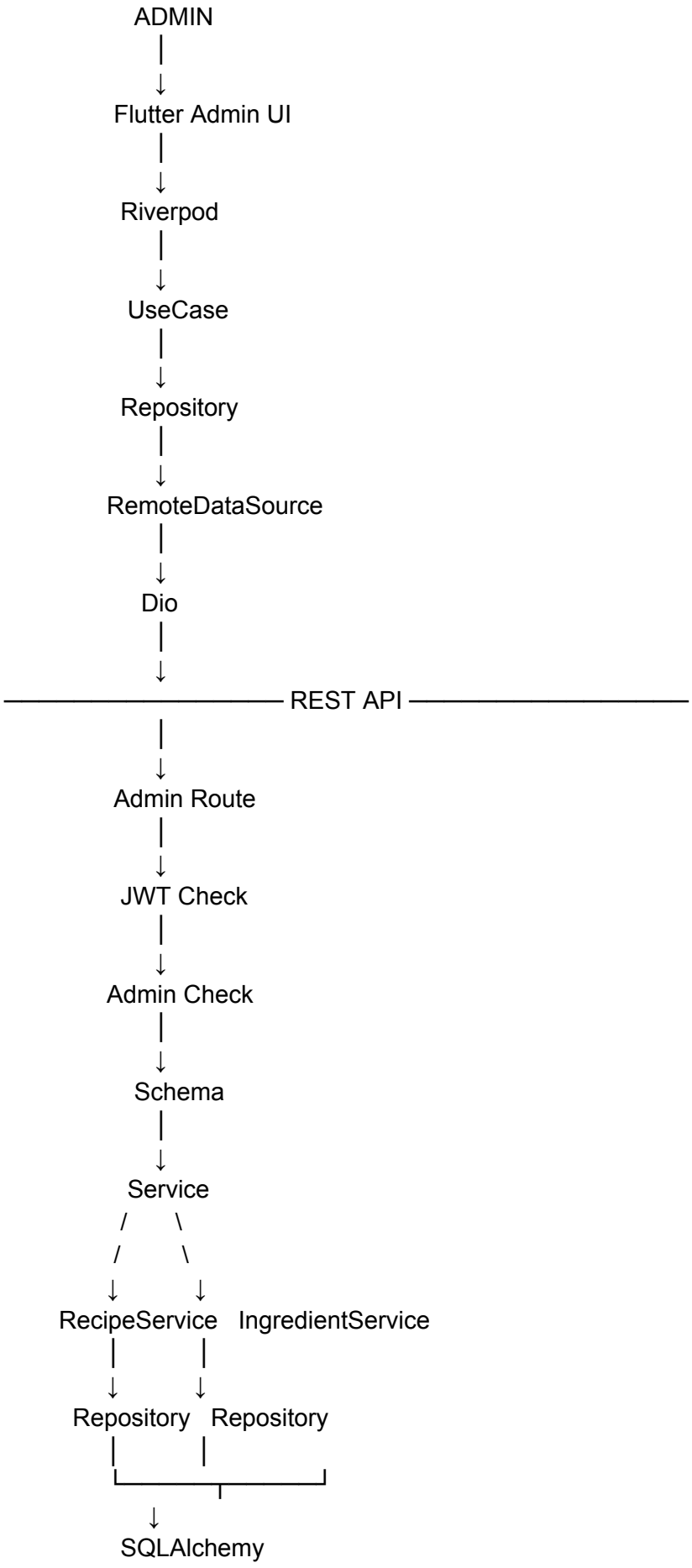
Her endpoint için:

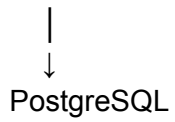
- Authentication
- Authorization
- Validation
- Success response
- Error response
- Database state

kontrol edilmelidir.

---

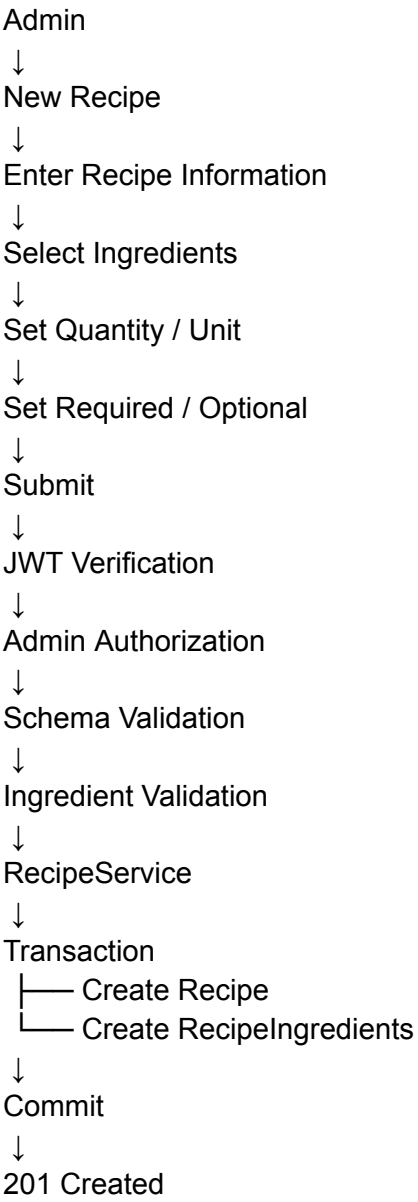
78. Admin Modülünün Genel Veri Akışı


ADMIN
Flutter Admin UI
Riverpod
UseCase
Repository
RemoteDataSource
Dio
REST API
Admin Route
JWT Check
Admin Check
Schema
Service
RecipeService
IngredientService
Repository
Repository
SQLAlchemy



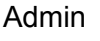
---

79. Recipe Creation Genel Akışı



---

80. Recipe Update Genel Akışı



↓
Edit Recipe
↓
Update Fields
↓
Update Ingredients
↓
Submit
↓
Validation
↓
Transaction BEGIN
↓
Update Recipe
↓
Replace RecipeIngredients
↓
COMMIT
↓
Success

Hata:

Validation / DB Error
↓
ROLLBACK
↓
Error Response

---

81. Ingredient Creation Genel Akışı

Admin
↓
Create Ingredient
↓
name
↓
normalize
↓
duplicate check
↓
create
↓
commit

↓
response

---

82. Recommendation Engine ile Admin Modülü Arasındaki Bağlantı

Admin modülü Recommendation Engine'in algoritmasını değiştirmez.

Admin yalnızca:

Recipe Data
Ingredient Data
RecipeIngredient Relationships

üretir.

Recommendation Engine:

Database
↓
Active Recipes
↓
Recipe Ingredients
↓
User Ingredients
↓
Matching
↓
Ranking

işlemini gerçekleştirir.

Bu ayrım sistemin maintainability açısından önemlidir.

---

83. Admin Modülünde AI Kullanımı

MVP'de admin recipe oluştururken AI otomatik olarak:

recipe generation

ingredient extraction
category generation

yapmayacaktır.

Admin veriyi kontrol ederek girecektir.

İleride AI destekli yardımcı özellikler eklenebilir:

Recipe text
↓
LLM
↓
Suggested ingredients
↓
Admin review
↓
Admin approval
↓
Database

Ancak AI tarafından üretilen içerik doğrudan production database'e yazılmamalıdır.

Admin onayı gereklidir.

---

84. Future Bulk Import

İleride admin:

CSV
JSON
Excel

üzerinden toplu recipe/ingredient aktarımı yapabilir.

Ancak import işlemi:

Upload
↓
Parse
↓
Validate
↓

Preview
↓
Admin Confirm
↓
Transaction
↓
Import

şeklinde tasarlanmalıdır.

Hatalı bir kaydın bütün veri setini kontrolsüz şekilde bozması engellenmelidir.

---

85. MVP İçin Önerilen Admin Ekranları

MVP'de yalnızca:

1. Admin Dashboard
2. Recipe List
3. Recipe Create
4. Recipe Edit
5. Ingredient List
6. Ingredient Create
7. Ingredient Edit

yeterlidir.

Ayrı bir:

Analytics Dashboard
User Management
Reports

ekranı zorunlu değildir.

---

86. Admin Modülünde UI Önceliği

Admin ekranlarında görsel tasarımdan çok:

Data accuracy

Validation
Usability
Error visibility

öncelikli olmalıdır.

Özellikle Recipe Form uzun olduğu için form bölümlere ayrılabilir:

1. Temel Bilgiler
2. Görsel ve Kaynak
3. Talimatlar
4. Malzemeler
5. Yayın Durumu

---

87. Admin Recipe Form Bölümleri

Bölüm 1 — Temel Bilgiler

Title
Description
Category

Bölüm 2 — Media / Source

Image URL
Source URL
YouTube URL

Bölüm 3 — Instructions

Instructions

Bölüm 4 — Ingredients

Ingredient
Quantity
Unit
Display Quantity
Optional

Bölüm 5 — Status

Active / Inactive

---

88. Admin Modülü İçin Son Mimari Kurallar

1. Admin yetkisi backend tarafından doğrulanır.
2. Frontend admin kontrolü yalnızca UI/UX amaçlıdır.
3. Recipe CRUD işlemleri transaction güvenli olmalıdır.
4. Ingredient CRUD işlemleri duplicate kontrolüne sahip olmalıdır.
5. `normalized_name` backend tarafından oluşturulur.
6. Semantic ingredient variants otomatik birleştirilmez.
7. RecipeIngredient ilişkileri kontrollü oluşturulur.
8. Aynı recipe içerisinde aynı ingredient tekrar edemez.
9. Quantity negatif olamaz.
10. Recipe en az bir required ingredient içermelidir.
11. Pasifleştirme fiziksel silmeye tercih edilir.
12. Inactive recipe recommendation sisteminden çıkarılır.
13. Inactive ingredient yeni ilişkilendirmelerde kullanılmaz.
14. Mevcut favorite/cart verileri otomatik olarak silinmemelidir.
15. Recipe ID güncelleme sırasında değişmemelidir.
16. Repository yalnızca database erişiminden sorumludur.
17. Service business logic'ten sorumludur.
18. Route business logic içermez.
19. Schema validation request yapısını kontrol eder.
20. AI-generated data doğrudan production'a yazılmamalıdır.
21. Admin işlemlerinde hassas bilgiler loglanmamalıdır.
22. Tüm admin endpointleri authentication + authorization gerektirir.

---

89. Document 13 Sonuç

Smart Chef & Pantry AI'nin tarif ve malzeme verileri recommendation sisteminin temel girdisini oluşturduğu için Admin Management modülü sistemin veri kalitesi açısından kritik bir bileşenidir.

Mimari olarak:

Admin
↓
Admin UI
↓

REST API
↓
Authorization
↓
Schema Validation
↓
Service
↓
Repository
↓
PostgreSQL

akışı korunmalıdır.

Recipe tarafında:

Recipe
+
RecipeIngredient
+
Ingredient

ilişkisi transaction içerisinde yönetilmelidir.

Ingredient tarafında:

name
↓
normalized_name
↓
duplicate check
↓
database

akışı kullanılmalıdır.

Admin tarafından oluşturulan veya güncellenen aktif recipe ve ingredient verileri, Recommendation Engine tarafından doğrudan kullanılabilir.

Özellikle MVP'de:

Admin
→ kontrollü veri girişi
→ deterministic recipe matching
→ kullanıcıya öneri

zinciri tercih edilmelidir.

Böylece sistemin veri katmanı ile AI/recommendation katmanı birbirinden ayrılmış olur ve ileride NLP, LLM, semantic ingredient matching veya personalized recommendation özellikleri eklenirken mevcut mimarinin bozulması engellenir.